

Arquitectura y Sistemas Operativos

Trabajo Práctico Integrador (TPI)

Investigación de Virtualización con VirtualBox y KVM

Alumnos

Héctor Atilio Signoriello - Comisión 10

Gregorio Agustin Hernandez - Comisión 12

Docentes:

Martín Aristiaran

Tutores:

Santos Sanchez

Andrés Odiard

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

25 de Junio de 2026

Tabla de contenido

Introducción	3
Marco Teórico	3
Caso Práctico	6
Metodología Utilizada	9
Resultados Obtenidos	10
Conclusiones	10
Bibliografía	11
Link	11

Introducción:

La virtualización ha sido seleccionada como tema de estudio debido a su practicidad a la hora de permitir testear herramientas o entornos para su estudio o implementación en el ámbito laboral y académico.

La virtualización es una tecnología fundamental en el ámbito de la informática moderna, la misma nos permite generar mediante software especializado denominado Hypervisor múltiples entornos independientes sobre un mismo hardware físico, optimizando el aprovechamiento de los recursos disponibles.

El presente trabajo práctico tiene como objetivo analizar los conceptos fundamentales de la virtualización, comprender su funcionamiento, identificar sus principales ventajas y limitaciones, y la exploración de diferentes tecnologías y herramientas utilizadas para su implementación. asimismo se busca adquirir conocimientos prácticos sobre la creación y administración de máquinas virtuales.

Marco Teórico:**Virtualización**

La virtualización es una tecnología que permite crear una representación virtual de recursos físicos, como servidores, sistemas operativos, dispositivos de almacenamientos o redes. Mediante esta técnica, un único equipo físico puede ejecutar múltiples máquinas virtuales (VM), cada una con su propio sistema operativo y aplicaciones, funcionando de manera aislada e independiente.

El componente principal de la virtualización es el hypervisor, software encargado de administrar los recursos de hardware y distribuirlos entre las máquinas virtuales. Los hypervisors pueden clasificarse en dos categorías: tipo 1 o nativos, que se ejecutan directamente sobre el hardware, y tipo 2 o alojados, que funcionan sobre un sistema operativo anfitrión.

Entre las principales ventajas de la virtualización se encuentra el mejor aprovechamiento de los recursos de hardware, la reducción de costos, la facilidad para realizar pruebas y la simplificación de tareas de administración y recuperación ante fallos.

Hypervisors:

Tipo 1: Un hypervisor tipo 1, también conocido como hypervisor nativo o bare metal, se ejecuta directamente en el hardware del host para administrar los sistemas operativos guest. Reemplaza a un sistema operativo host y los recursos de VM se programan directamente en el hardware mediante el hypervisor.

Este tipo de hypervisor es más común en un centro de datos empresarial u otros entornos basados en servidores.

KVM, Microsoft Hyper-V y VMware vSphere son ejemplos de un hipervisor tipo 1. KVM se fusionó con el kernel de Linux en 2007, por lo que si estás usando una versión moderna de Linux, ya tienes acceso a KVM

Tipo 2: Un hypervisor tipo 2 también se conoce como hypervisor alojado (hosted), y se ejecuta en un sistema operativo convencional como capa de software o aplicación.

Funciona abstrayendo los sistemas operativos invitados (guest) del sistema operativo host. Los recursos de VM se estructuran sobre el SO host, que luego se ejecuta contra el hardware.

Un hypervisor tipo 2 es mejor para usuarios individuales que desean ejecutar múltiples sistemas operativos en una computadora personal.

VMware Workstation y Oracle VirtualBox son ejemplos de un hypervisor tipo 2.

VirtualBox:

VirtualBox es un hypervisor de tipo 2 desarrollado por Oracle que permite crear y administrar máquinas virtuales sobre diversos sistemas operativos anfitriones, como Windows, Linux y macOS. Su interfaz gráfica facilita la configuración de recursos como memoria RAM, procesadores, Almacenamiento y redes virtuales.

Esta herramienta es ampliamente utilizada en entornos educativos, de desarrollo y pruebas debido a su facilidad de uso y compatibilidad con múltiples sistemas operativos. Además, permite la creación de instantáneas (snapshots), que facilitan la restauración de una máquina virtual a un estado anterior.

KVM(Kernel-based Virtual Machine):

KVM es una tecnología de virtualización integrada en el núcleo de Linux que convierte al sistema operativo en un hypervisor de tipo 1. Aprovecha las extensiones de virtualización por hardware presentes en los procesadores modernos, como Intel VT-X y AMD-V, para ofrecer un rendimiento cercano al de una máquina física.

A diferencia de VirtualBox, KVM está orientado principalmente a entornos profesionales y servidores, donde se requiere una mayor eficiencia y escalabilidad. La administración de las máquinas virtuales puede realizarse mediante herramientas como Virt-Manager, gnome Boxes o a través de la línea de comandos utilizando libvirt.

Modos de Red en Máquinas virtuales

La conectividad de red es un aspecto fundamental en los entornos virtualizados, ya que permite la comunicación entre las máquinas virtuales, el sistema anfitrión y otras redes externas. Los hypervisors ofrecen diferentes modos de redes para adaptar el comportamiento de las máquinas virtuales según las necesidades de cada entorno.

NAT (Network Address Translation)

El modo NAT permite que una máquina virtual acceda a redes externas, como Internet, utilizando la dirección IP del equipo Host. En esta configuración, el hypervisor actúa como intermediario entre la máquina virtual y la red física, traduciendo las direcciones de red para que el tráfico pueda ser enviado y recibido correctamente.

Este modo es uno de los más utilizados debido a su simplicidad de configuración y porque proporciona acceso a Internet sin requerir cambios en la infraestructura de red existente. Sin embargo, las conexiones entrantes desde otros equipos hacia la máquina virtual suelen requerir configuraciones adicionales de redirección de puertos.

Caso Práctico

El caso práctico se encuentra dividido en dos partes. La primera parte consiste en la creación y configuración de una máquina virtual mediante VirtualBox, un hypervisor de tipo 2, en la cual se instaló el sistema operativo Ubuntu. Dentro de dicha máquina virtual se desarrolló y ejecutó un programa en Python que convierte un número decimal ingresado por el usuario a su equivalente en sistema binario, utilizado como caso de aplicación práctica de la virtualización.

La segunda parte consiste en instalación de los componentes necesarios para KVM/QEMU, un hypervisor de tipo 1, en un sistema operativo Linux distribución fedora y las configuraciones necesarias para gestionar una conexión de red en modo Bridge, continuando con la creación de una VM con ubuntu server, configuración necesaria para la importación del repositorio del TPI y la correcta ejecución del script conversor.py

parte 1:

1. se obtuvo el instalador para VirtualBox desde la página oficial <https://www.virtualbox.org/wiki/Downloads>
2. Se procedió a la instalación de VirtualBox.
3. Se descargó la imagen ISO de Ubuntu desde la página oficial <https://ubuntu.com/download>
4. Se creó una nueva máquina virtual en VirtualBox, seleccionando la imagen ISO de Ubuntu como medio de instalación, y se la configuró con 4 núcleos de procesador y 4090 MB de memoria RAM con el objetivo de optimizar su velocidad de ejecución.
5. Se inició la máquina virtual y se completó la instalación de Ubuntu siguiendo el asistente del sistema operativo.
6. Desde la terminal de Ubuntu se verificó la versión de Python instalada con el comando `python3 --version`.
7. Se creó, mediante el editor nano, el archivo conversor.py con el código del programa de conversión de decimal a binario.
8. Se ejecutó el programa con el comando `python3 conversor.py`, se ingresó un número decimal y se verificó que el resultado en binario mostrado en pantalla fuera correcto.

parte 2:

1. Se verifica que la computadora sea compatible con la virtualización mediante los siguientes comandos.
 - a. `lscpu | grep -i virtualization`
 - b. `zgrep CONFIG_KVM /boot/config-$(uname -r)`
2. De ser compatible se procede a la instalación de las dependencias de KVM ya que el mismo se encuentra instalado por defecto en el Kernel de Linux.
 - a. `sudo dnf install qemu-kvm libvirt virt-install virt-manager virt-viewer edk2-ovmf swtpm
qemu-img guestfs-tools libosinfo tuned`
3. Proseguimos con la habilitación del daemon libvirt de forma modular.
 - a. `for drv in qemu interface network nodedev nwfilter secret storage; do \
sudo systemctl enable virt${drv}d.service; \
sudo systemctl enable virt${drv}d{,-ro,-admin}.socket; \
done`
4. Se comprueba que la instalación sea correcta.
 - a. `sudo virt-host-validate qemu`
5. Continuamos con la configuración de la conexión de la red en modo Bridge.
 - a. Verificamos los dispositivos de red disponibles en el Host.
 - `sudo nmcli device status`
 - b. Creamos la interfaz Bridge bridge0.
 - `sudo nmcli connection add type bridge con-name bridge0 ifname bridge0`
 - c. Asignamos la interfaz bridge a nuestro dispositivo (enp4s0 debe ser reemplazado con la interfaz correspondiente).
 - `sudo nmcli connection add type ethernet slave-type bridge \
con-name 'Bridge connection 1' ifname enp4s0 master bridge0`
 - d. Activamos la conexión
 - `sudo nmcli connection up bridge0`
 - e. Habilitamos la autoconexión y reactivamos la conexión.
 - `sudo nmcli connection modify bridge0 connection.autoconnect-slaves 1`

- `sudo nmcli` connection up bridge0
- f. Comprobamos que la conexión se encuentra establecida.
 - `sudo nmcli` device status
 - `ip -brief addr show dev bridge0`
- 6. Terminada la configuración del Host tanto en el área de hypervisor como en la red procedemos a descargar la ISO de nuestro sistema operativo a instalar, en este caso Ubuntu server 26.04 LTS.
<https://ubuntu.com/download/server>
- 7. Continuamos con la creación de la VM con nuestra GUI virt-manager, asignado 2 núcleos de CPU y 4096 MiB de RAM, creamos un almacenamiento de 25 GiB y asignamos la interfaz de red Bridge0 a la VM.
- 8. Iniciamos la instalación y completamos los pasos necesarios para la misma,
 - a. Selección de idioma
 - b. Configuración de distribución de teclado
 - c. Selección de tipo de instalación (normal, mínima o instalación de paquetes de terceros)
 - d. Configuración de red y verificación de la misma
 - e. Particionado del almacenamiento
 - f. Configuración de usuario y contraseña
 - g. Se aprovecha la opción de instalar OpenSSH en el wizard de instalación.
- 9. Continuamos con la puesta a punto de nuestro SO.
 - a. Sincronizamos los paquetes y actualizamos el servidor
 - `sudo apt update && sudo apt upgrade`
 - b. Verificamos de tener los paquetes necesarios python3 y git instalados en el sistema
 - `python3 --version`
 - `git --version`
- 10. Configuramos nuestra clave ssh y git para poder clonar nuestro repositorio
 - a. `ssh-keygen -t ed25519 -C "your\_email@example.com"`
 - b. `git config --global user.name "Your Name"`

- c. `git config --global user.email "your.email@example.com"`
- 11. Debemos agregar nuestra clave ssh en github para poder continuar
- 12. Clonamos nuestro repositorio donde se aloja el script conversor.py a ejecutar
 - a. `git clone git@github.com:ghernanandez/tpi-virtualizacion.git`
- 13. Nos movemos a la carpeta tpi-virtualizacion y ejecutamos el script
 - a. `cd tpi-virtualizacion`
 - b. `python3 conversor.py`

Metodología utilizada

El trabajo se desarrolló siguiendo un enfoque práctico, combinando la investigación teórica sobre virtualización con la implementación de un caso concreto.

En primer lugar se relevó información sobre los conceptos fundamentales de virtualización y hypervisors a partir de fuentes oficiales y documentación técnica tales como RedHat, VMware, Oracle para la investigación de los hypervisor y los distintos tipos que hay, las distintas tipos de conexiones de red Bridge y Nat que se pueden encontrar en la configuración tanto del Host como de guest, además de consultas técnicas como la configuración de git y la generación de claves ssh para la correcta configuración del entorno

Luego en la parte 1 se procedió a la instalación de VirtualBox y a la configuración de una máquina virtual con Ubuntu, optimizando sus recursos de hardware (4 núcleos y 4090 MiB de RAM) para garantizar un funcionamiento ágil. Finalmente, dentro de dicho entorno se desarrolló, ejecutó y validó un programa en Python como caso de aplicación práctica, documentando cada paso mediante capturas de pantalla.

Para continuar en la parte 2 se instaló y configuró el entorno de virtualización KVM/QEMU en Linux con su respectiva modificación del adaptador de red para poder usar una conexión Bridge en la VM. Seguido de eso se creó una VM con virt-manager en la cual se le asignó recursos mínimos para su correcto funcionamiento (2 núcleos, 4096 MiB de RAM y 25 GiB de almacenamiento) y se le asignó la interfaz en modo Bridge.

Para terminar se realizó la instalación de sistema operativo y se configuro todo su entorno para la posterior clonación del repositorio y la ejecución del script `conversor.py`

Resultados Obtenidos

En la parte 1 se logró instalar y configurar correctamente VirtualBox junto con una máquina virtual con Ubuntu, lo que permitió un funcionamiento ágil del sistema operativo invitado. Dentro de este entorno se ejecutó satisfactoriamente el programa en Python, obteniéndose conversiones correctas de distintos números decimales a su equivalente binario. Las capturas de pantalla incluidas en los Anexos evidencian tanto la configuración de la máquina virtual como la ejecución exitosa del programa.

Para la parte 2 se logró la correcta instalación del entorno KVM/QEMU con la correcta configuración de la interfaz de red habilitada para su uso en modo Bridge, de esa forma se pudo proseguir con la creación de la VM e instalación del sistema operativo Ubuntu server, garantizando un consumo mínimo del Guest, lo cual permite un correcto funcionamiento del mismo y el Host, por otro lado se pudo configurar de manera satisfactoria el entorno para la posterior clonación del repositorio y realiza la ejecución del script.

Como se pueden observar en las capturas de configuración del Host el mismo no posee la habilidad de gestionar asignación IOMMU ya que no es una tecnología soportada por el CPU ni por la placa de la computadora, limitando las capacidades de la misma ya que no permite un passthrough de dispositivos tales como una placa de red o GPU

Conclusiones

El desarrollo de este trabajo práctico permitió comprender en profundidad los conceptos fundamentales de la virtualización y su utilidad para crear entornos de prueba aislados sobre un mismo equipo físico. La implementación práctica con VirtualBox , KVM y Ubuntu demostró que es posible configurar una máquina virtual ajustando sus recursos de hardware para optimizar su rendimiento, y utilizarla como entorno de desarrollo y ejecución de software. Asimismo, se reforzaron conocimientos de programación en Python al desarrollar un programa funcional dentro de dicho entorno. En conjunto, el trabajo cumplió con los objetivos propuestos, brindando una experiencia integral que combina teoría y práctica sobre arquitectura y sistemas operativos.

Bibliografía / Webgrafía

Red Hat. (2023). What is a hypervisor?

<https://www.redhat.com/en/topics/virtualization/what-is-a-hypervisor>

VMware. (s. f.). What is a hypervisor?

<https://www.vmware.com/topics/hypervisor>

Oracle. (s. f.). Introduction to Oracle VirtualBox.

<https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/Introduction.html>

ArchWiki. (s. f.). KVM.

<https://wiki.archlinux.org/title/KVM>

Chacon, S., & Straub, B. (s. f.). Configurando Git por primera vez. En Pro Git (2.ª ed.). Git.

<https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Configurando-Git-por-primera-vez>

Oracle. (s. f.). Virtual networking. En Oracle VirtualBox User Manual.

<https://www.virtualbox.org/manual/topics/networkingdetails.html#networkingdetails>

Canonical Ltd. (s. f.). Download Ubuntu Desktop.

<https://ubuntu.com/download/desktop>

Chacon, S., & Straub, B. (s. f.). Generando tu clave pública SSH. En Pro Git (2.ª ed.). Git.

<https://git-scm.com/book/es/v2/Git-en-el-Servidor-Generando-tu-clave-p%C3%BAblica-SSH>

Link:

Hernandez Gregorio Agustin, Signoriello Héctor Atilio (2026). Virtualización con VirtualBox y KVM [Código fuente]. GitHub.

<https://github.com/ghernandez/tpi-virtualizacion>

Hernandez Gregorio Agustin, Signoriello Héctor Atilio (2026). Virtualización con VirtualBox y KVM [Vídeo]. Youtube.

<https://www.youtube.com/watch?v=LXr8kYkU2GU>